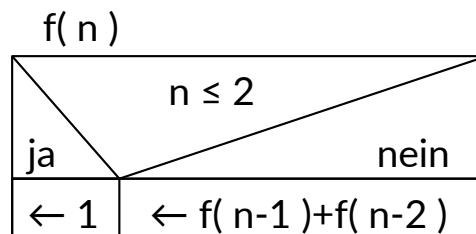


Algorithmen und Datenstrukturen

Aufgabe 1: Iterativ vs. rekursiv

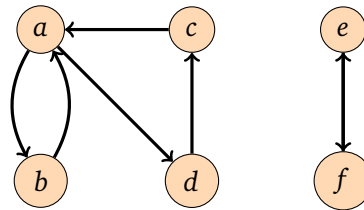
Gegeben ist folgender Algorithmus $f(n)$.



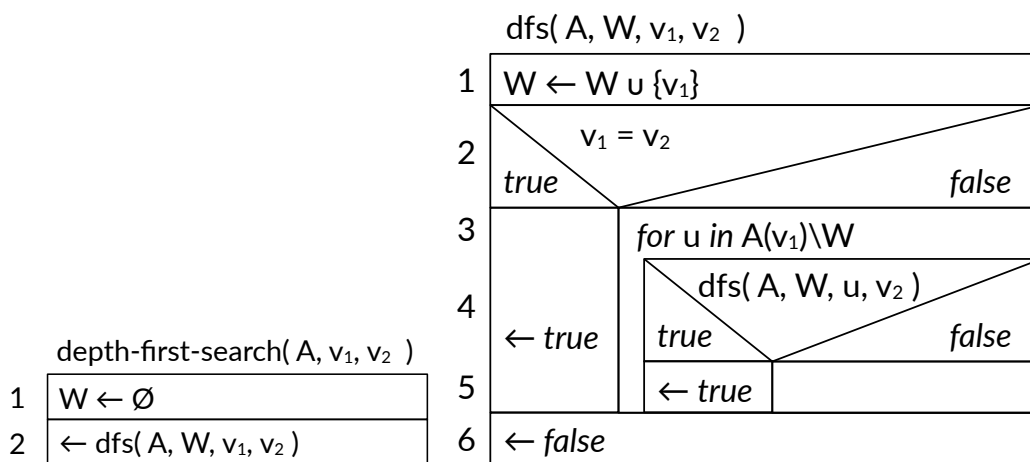
- a). Welches Programmierparadigma (iterativ oder rekursiv) verwendet der Algorithmus?
- b). Vollziehe händisch die Berechnung der Werte $f(1), f(2), f(3), f(4), f(5)$ nach. Welche Funktion implementiert der Algorithmus?
- c). Entwirf einen Algorithmus, der die gleiche Funktion im entgegengesetzten Programmierparadigma implementiert. Gib ihn als Struktogramm an.
- d). Schätze die Zeitkomplexität des gegebenen Algorithmus $f(n)$ mit einem geschlossenen Ausdruck nach oben ab! Nutze dazu eine Rekursionsgleichung!
- e). Was ist die Zeitkomplexität deines Algorithmus?
- f). Implementiere beide Algorithmen und verifiziere das Laufzeitverhalten.
- g). Gibt es eine Möglichkeit, den Algorithmus $f(n)$ ohne Änderung des Programmierparadigmas so umzuformulieren, dass seine Worst-Case-Laufzeit in $\mathcal{O}(n)$ ist?

Aufgabe 2: Breiten und Tiefensuche

Gegeben ist der folgende Graph $G = (V, E)$.



- Identifiziere die starken Zusammenhangskomponenten des Graphen.
- Mache dich mit dem Tiefensuche-Algorithmus vertraut, der durch das folgende Struktogramm repräsentiert wird:



Nutze Tiefensuche, um einen Algorithmus zu gestalten, der ermittelt, ob zwei beliebige Knoten von G Teil derselben starken Zusammenhangskomponente sind. Implementiere deinen Algorithmus unter Verwendung der Funktion `tiefensuche` im Jupyter-Notebook und teste ihn für die Knoten a und c .

- Erweitere deinen Algorithmus, sodass er nur noch **einen** Knoten als Eingabe hat und alle Knoten, die sich in derselben Zusammenhangskomponente befinden, ermittelt. Implementiere deinen Algorithmus im Jupyter-Notebook und teste ihn für die dort aufzufindenden Graphen.
- Ändere deinen Algorithmus so ab, dass er iterativ ist und auf Breitensuche statt auf Tiefensuche basiert.